Expected Goal Threat as an A* heuristic for the Romania map

Recommended setting: β = 0.02, γ = 0.01, τ = 1.5. Every number in this document is computed by `make_pdf.py` from the same code that produces the exported tables.

This document has four parts: (1) what xT means in real football, (2) how the same idea is applied to the Romania road map, (3) every variable explained, and (4) the calculation step by step with the full Pitesti example worked out column by column. Part 6 follows a whole search, Zerind → Bucharest, through two forks in the road; Part 7 shows the full table and the parameter choices.

Part 1 — What "Expected Threat" means in real football

Expected Threat (xT) is a football-analytics model (Karun Singh, 2018) that answers one question: **"If my team has the ball at this spot on the pitch, how likely is it that this possession ends in a goal?"** It exists because "distance to goal" is a poor measure of danger: a ball on the wing 20 m from goal is less dangerous than a ball in the centre 25 m out, because of what the player can *do next* from each spot.

How the real model works:

1. The pitch is cut into a grid of zones (typically 12 × 8 = 96 zones).

2. From thousands of real matches, for every zone the analysts count what players actually did with the ball there: how often they **shot**, how often they **moved the ball** (pass or dribble), and — when they moved it — **which zone it went to**. Those counts become probabilities: shoot probability, move probability, and a transition probability from each zone to each other zone. From the shots they also get the chance that a shot from that zone scores (xG).

3. The value of a zone is then defined by looking one action ahead:

```
xT(zone) = (chance of shooting here) × (chance the shot scores)
         + (chance of moving the ball) × Σ over target zones [ P(zone → target) × xT(target)
]
```

The second line is the important idea: **a zone is worth a lot if the zones you can reach from it are worth a lot.** The definition is circular (xT on both sides), so it is solved by iteration: start with xT = 0 everywhere, apply the formula to every zone, repeat 5–10 times; the numbers settle. The result is a heat map where the value rises smoothly toward the goal mouth, but not uniformly — central zones with good onward options score higher than wide zones at the same distance.

4. Analysts then grade an action by the threat it *added*: xT(end zone) − xT(start zone). A pass that moves the ball from a 0.05 zone to a 0.20 zone "created 0.15 xT".

Three ideas we keep: (a) value comes from the future actions available, not from distance; (b) that value is propagated backwards from the goal through the network of possible moves; (c) it is computed by simple repeated iteration of one formula.

Part 2 — Applying the same idea to the Romania map

Real football (xT)	Romania map (xGT)
a zone of the pitch	a city
a pass or dribble from zone to zone	driving along one road from city to city
transition probabilities measured from match data	transition probabilities **computed from road cost** — a short road is an easy pass, a long road is a hard pass
a shot that scores	arriving at the goal city (value 1)
losing the ball (pass fails)	a pass along a long road may "fail" — the possession is lost, value 0

xT(zone) = chance this possession ends in a goal	xGT(city) = chance a possession starting here reaches the goal city
heat map used to grade passes	value turned into an integer h(city) for A*

Two things have to be invented because the map has no match data:

• **Which road does the "player" take?** In real xT this comes from data. Here we model it: the player prefers short roads (easy passes) and prefers roads that lead to cities which are themselves threatening — that second preference is what we call the **value-aware passer**. Without it the player picks roads by cost alone, and a city with many roads spreads its probability over bad options and looks worse than it is (the "dilution" problem).

• **Can the pass fail?** In real xT a possession can end without a shot. Here every road has a completion probability that falls with its cost; a failed pass ends the possession with value 0. Without this, every possession would eventually reach the goal and every city would get value 1 — the model would say nothing.

There is no shooting step: the only way to "score" is to arrive at the goal city, so xGT(goal) = 1 and the whole value field is propagated backwards from that one city.

Part 3 — Every symbol, explained

The confusing part of the formulas is the notation `(u, v)`. It always means **one road**: **u** is the city where the ball is now (the passer), **v** is the neighbouring city the ball would go to (the receiver). So `w(u,v)` is "the cost of the road from u to v", and `C(u,v)` is "the chance a pass from u to v is completed". When we write "sum over v" we mean: do it once for each road leaving u, then add the results.

symbol	what it is	example (goal = Bucharest)
goal	the target city; its value is fixed at 1	Bucharest
u	the city we are evaluating (where the ball is)	Pitesti
v	one neighbour of u (a possible receiver)	Bucharest, Rimnicu Vilcea or Craiova
w(u,v)	road cost between u and v (from the map)	w(Pitesti, Bucharest) = 101
β (beta) = 0.02	how strongly the passer prefers *cheap* roads	a road twice as long is about 7× less attractive
γ (gamma) = 0.01	how fast the *completion* chance drops with road cost	a 100-cost road is completed 37 % of the time
τ (tau) = 1.5	how strongly the passer prefers receivers that are themselves threatening	τ = 0: ignores it; larger τ: more goal-directed
$C(u,v) = e^{-\gamma \cdot w(u,v)}$	completion probability of the pass along road (u,v)	$C(\text{Pitesti, Bucharest}) = e^{-1.01} = 0.364$
$q(u,v) = e^{-\beta \cdot w(u,v)}$	how attractive the road is by cost alone	$q(\text{Pitesti, Bucharest}) = e^{-2.02} = 0.133$
P(u→v)	probability the passer at u chooses road (u,v); all P(u→·) add up to 1	P(Pitesti→Bucharest) = 0.948
xGT(u)	chance a possession starting at u eventually reaches the goal (0 … 1)	xGT(Pitesti) = 0.347
h(u)	the A* heuristic: xGT converted to an integer in road-cost units; 0 at the goal	h(Pitesti) = 106

Part 4 — How h(n) is calculated, step by step

Step 1 — completion probability of every road (done once)

```
C(u,v) = exp( −γ · w(u,v) )        γ = 0.01
```

Examples: road 85 → 0.427; road 101 → 0.364; road 140 → 0.247; road 211 → 0.121. A long road is a risky pass.

Step 2 — cost-attractiveness of every road (done once)

```
q(u,v) = exp( −β · w(u,v) )        β = 0.02
```

This is only the *cost* half of the passer's preference. It is not yet a probability.

Step 3 — which road the passer chooses (recomputed every iteration)

```
preference(u,v) = q(u,v) · xGT(v)^τ                τ = 1.5
P(u→v)          = preference(u,v) / Σ over all roads (u,k) of preference(u,k)
```

Read it as: "the passer likes a road if it is short **and** it leads somewhere threatening"; dividing by the sum turns the preferences into probabilities that add up to 1. Because xGT(v) is used here, step 3 depends on the values we are still computing — that is why the whole thing is iterated (step 5).

Step 4 — the value of a city

```
xGT(goal) = 1
xGT(u)    = Σ over all roads (u,v) of   P(u→v) · C(u,v) · xGT(v)
```

In words: for each road out of u, multiply (chance the passer picks it) × (chance the pass arrives) × (value of where it arrives), then add the roads up. Compare with the real xT formula in Part 1: it is the same “move probability × transition probability × value of target” term; the shooting term is replaced by xGT(goal) = 1.

Step 5 — iterate until the numbers stop changing

Start with xGT = 0.001 for every city and 1 for the goal. Apply steps 3–4 to all 20 cities; that is one iteration. Repeat until the largest change in any city is below 10^{-12} — this takes 34–40 rounds for every goal (a literal "nothing changed" test on doubles never fires for most goals, because the last bit keeps flipping; a tolerance is the correct stopping test). The table shows how the values settle for goal = Bucharest:

iteration	Giurgiu	Pitesti	Rimnicu Vilcea	Craiova	Sibiu	Arad
1	0.4066	0.3642	0.0004	0.0003	0.0004	0.0004
2	0.4066	0.3642	0.1381	0.0916	0.0450	0.0002
3	0.4066	0.3438	0.1266	0.0817	0.0559	0.0111
4	0.4066	0.3460	0.1175	0.0755	0.0510	0.0138
5	0.4066	0.3477	0.1196	0.0768	0.0477	0.0113
6	0.4066	0.3474	0.1207	0.0775	0.0486	0.0101
8	0.4066	0.3472	0.1202	0.0772	0.0489	0.0110
converged	0.4066	0.3472	0.1203	0.0772	0.0489	0.0108

Giurgiu is fixed after one round (its only road goes straight to the goal). Pitesti needs a few rounds because its value depends on Rimnicu Vilcea and Craiova, whose values depend on Sibiu and Drobeta, and so on outward. Values shrink with distance from the goal: each extra pass multiplies by a completion chance below 1.

Step 6 — turn the value into an integer heuristic

```
h(u) = floor( −ln( xGT(u) ) / γ  + 0.5 )          h(goal) = 0
```

Why −ln and why divide by γ: along one chain of passes the value is a product of factors $e^{-\gamma \cdot w}$, so −ln of it is γ × (sum of road costs). Dividing by γ gives back a number in road-cost units — the same units as g(n), which is what A* needs for f = g + h. High threat → low h; the goal has value 1 → h = 0. The +0.5 and floor just round to the nearest integer in a way that is identical in Python and Java.

Part 5 — The Pitesti example, column by column (goal = Bucharest)

Pitesti has **three roads**: to Bucharest (101), to Rimnicu Vilcea (97) and to Craiova (138). We evaluate Pitesti as **u**; each of the three cities is one **v**. The xGT values of the receivers come from the converged iteration (step 5): xGT(Bucharest) = 1 (it is the goal), xGT(Rimnicu Vilcea) = 0.1203, xGT(Craiova) = 0.0772.

receiver v	w(u,v)	q = e^(−0.02·w)	xGT(v)	xGT(v)^1.5	preference = q · xGT(v)^1.5	P(u→v) = pref / sum	C = e^(−0.01·w)	P · C · xGT(v)
Bucharest	101	0.1327	1.0000	1.0000	0.13266	0.9475	0.3642	0.3451
Rimnicu Vilcea	97	0.1437	0.1203	0.0417	0.00600	0.0428	0.3791	0.0020
Craiova	138	0.0633	0.0772	0.0215	0.00136	0.0097	0.2516	0.0002

How to read each column:

• **w(u,v)** — road cost from the map.

• **q** — step 2, cost-attractiveness: $e^{-0.02 \cdot 101}$ = 0.133 for the Bucharest road.

• **xGT(v)** — the receiver's value from the last iteration; **xGT(v)^1.5** is the same number raised to τ.

• **preference** — q × xGT(v)^1.5. The three preferences add up to 0.14001.

• **P(u→v)** — each preference divided by that sum. This is the probability the passer at Pitesti picks that road; the three P values add up to 1. The direct road to Bucharest gets 0.9475 because it is both short and leads to the goal.

• **C** — step 1, completion chance of that road.

• **P · C · xGT(v)** — the road's contribution to Pitesti's value (step 4).

```
xGT(Pitesti) = 0.3451 + 0.0020 + 0.0002 = 0.3472
h(Pitesti)   = floor( −ln(0.3472) / 0.01 + 0.5 ) = floor( 105.8 + 0.5 ) = 106
```

Almost all of Pitesti's value comes from the direct road (0.345 of 0.347). The two side roads contribute little, but the fact that the passer *sometimes* takes them is why h(Pitesti) = 106 is a bit above the true remaining cost 101 — that small surplus is the football content of the number.

The simplest case: Giurgiu

Giurgiu has one road (to Bucharest, 90). With one road the preference sum is the preference itself, so P = 1:

```
xGT(Giurgiu) = 1 · C(Giurgiu,Bucharest) · xGT(Bucharest) = 1 · 0.4066 · 1 = 0.4066
h(Giurgiu)   = floor( −ln(0.4066) / 0.01 + 0.5 ) = 90
```

A single road means no hesitation, so h equals the road cost exactly (90). That is the sanity check from the guide: h(Giurgiu) = 90 < h(Pitesti) = 106, matching true costs 90 < 101.

A three-road city again: Urziceni

receiver v	w(u,v)	q	xGT(v)	xGT(v)^1.5	preference	P(u→v)	C	P · C · xGT(v)
Bucharest	85	0.1827	1.0000	1.0000	0.18268	0.9510	0.4274	0.4065
Hirsova	98	0.1409	0.1447	0.0550	0.00775	0.0404	0.3753	0.0022
Vaslui	142	0.0584	0.0934	0.0285	0.00167	0.0087	0.2417	0.0002

```
xGT(Urziceni) = 0.4065 + 0.0022 + 0.0002 = 0.4088   →   h(Urziceni) = 89   (true cost 85)
```

Part 6 — A whole search with forks: Zerind → Bucharest

Pitesti shows how one city's number is built, but the interesting question for A* is what happens at a **fork**: two roads leave a city, both eventually reach the goal, and the cheaper first step is *not* the cheaper route. The guide's own example is exactly this. From Zerind (goal = Bucharest):

```
Zerind → Arad   (75) → Sibiu (140) = 215 to Sibiu   ← cheaper route, dearer first road
Zerind → Oradea (71) → Sibiu (151) = 222 to Sibiu   ← cheaper first road, dearer route

then from Sibiu, a second fork:
Sibiu → Rimnicu Vilcea (80) → Pitesti (97) → Bucharest (101)   = 278   ← cheaper
Sibiu → Fagaras (99)        → Bucharest (211)                  = 310

best: Zerind → Arad → Sibiu → Rimnicu Vilcea → Pitesti → Bucharest = 493  (via Oradea: 500)
```

A good heuristic must therefore say h(Arad) < h(Oradea) even though Oradea is the cheaper first step, and h(Rimnicu Vilcea) < h(Fagaras). Here is how the model arrives at both.

Fork 1 — Arad versus Oradea (both evaluated toward Bucharest)

Both cities' value comes almost entirely from their road to Sibiu (xGT(Sibiu) = 0.0489); the back-road to Zerind and Arad's road to Timisoara lead to low-value cities and contribute almost nothing. Same columns as the Pitesti table:

Arad — 3 roads, preference sum 0.000750:

receiver v	w(u,v)	q	xGT(v)	xGT(v)^ 1.5	preference	P(u→v)	C	P · C · xGT(v)
Sibiu	140	0.0608	0.0489	0.0108	0.00066	0.8770	0.2466	0.0106
Zerind	75	0.2231	0.0049	0.0003	0.00008	0.1033	0.4724	0.0002
Timisoara	118	0.0944	0.0029	0.0002	0.00001	0.0196	0.3073	0.0000

```
xGT(Arad) = 0.0106 + 0.0002 + 0.0000 = 0.0108   →   h(Arad) = floor(-ln(0.0108)/0.01 + 0.5) =
 453
```

Oradea — 2 roads, preference sum 0.000612:

receiver v	w(u,v)	q	xGT(v)	xGT(v)^ 1.5	preference	P(u→v)	C	P · C · xGT(v)
Sibiu	151	0.0488	0.0489	0.0108	0.00053	0.8628	0.2209	0.0093
Zerind	71	0.2417	0.0049	0.0003	0.00008	0.1372	0.4916	0.0003

```
xGT(Oradea) = 0.0093 + 0.0003 = 0.0097   →   h(Oradea) = floor(-ln(0.0097)/0.01 + 0.5) = 464
```

So h(Arad) = 453 < h(Oradea) = 464. The 11-point gap comes from one number: the completion probability of the road to Sibiu, C = 0.2466 for Arad's 140-road versus 0.2209 for Oradea's 151-road. The model does not know the route costs 418 and 429 — it only knows that Arad's pass to Sibiu is a bit easier to complete — and that is enough to reproduce the true difference (429 − 418 = 11).

Fork 2 — Rimnicu Vilcea versus Fagaras (seen from Sibiu)

receiver v	w(u,v)	q	xGT(v)	xGT(v)^ 1.5	preference	P(u→v)	C	P · C · xGT(v)
Rimnicu Vilcea	80	0.2019	0.1203	0.0417	0.00842	0.6151	0.4493	0.0332
Fagaras	99	0.1381	0.1117	0.0373	0.00516	0.3765	0.3716	0.0156
Oradea	151	0.0488	0.0097	0.0009	0.00005	0.0034	0.2209	0.0000

Arad	140	0.0608	0.0108	0.0011	0.00007	0.0050	0.2466	0.0000

Sibiu's passer prefers Rimnicu Vilcea (P = 0.6151) over Fagaras (P = 0.3765): the road is shorter and Rimnicu Vilcea is slightly more threatening (xGT 0.1203 vs 0.1117, because 97 + 101 with a stop at Pitesti completes more often than one 211 pass). Individually, h(Rimnicu Vilcea) = 212 < h(Fagaras) = 219 — true costs 198 vs 211.

Running A* from Zerind with these h values

A* always expands the frontier city with the smallest f = g + h (g = road cost already paid). Every step of the search:

step	expanded (g + h = f)	frontier after expansion, sorted by f
1	**Zerind** (0 + 531 = 531)	Arad (75+453=528), Oradea (71+464=535)
2	**Arad** (75 + 453 = 528)	Sibiu (215+302=517), Oradea (71+464=535), Timisoara (193+584=777)
3	**Sibiu** (215 + 302 = 517)	Rimnicu Vilcea (295+212=507), Fagaras (314+219=533), Oradea (71+464=535), Timisoara (193+584=777)
4	**Rimnicu Vilcea** (295 + 212 = 507)	Pitesti (392+106=498), Fagaras (314+219=533), Oradea (71+464=535), Craiova (441+256=697), Timisoara (193+584=777)
5	**Pitesti** (392 + 106 = 498)	Bucharest (493+0=493), Fagaras (314+219=533), Oradea (71+464=535), Craiova (441+256=697), Timisoara (193+584=777)
6	**Bucharest** (493 + 0 = 493) — goal reached	—

Result: path **Zerind** → **Arad** → **Sibiu** → **Rimnicu Vilcea** → **Pitesti** → **Bucharest**, cost **493** (the optimum), **6 cities expanded** — uniform-cost search needs 13 for the same pair, straight-line distance 8, the old ring method 8.

Look at step 1: f(Arad) = 75 + 453 = 528 beats f(Oradea) = 71 + 464 = 535. The 4-point head start of Oradea's cheaper road is outweighed by the 11-point heuristic difference, so A* goes through Arad immediately. Oradea stays on the frontier the whole way (f = 535) and is never expanded — correctly, since no route through it can beat 493. At step 3 the same happens for the second fork: Rimnicu Vilcea (295 + 212 = 507) beats Fagaras (314 + 219 = 533). Nothing was expanded that is not on the final path.

What the cost-only passer (τ = 0) does at the same fork

With the pass choice depending on road cost only (β = γ = 0.01), the same fork gives h(Arad) = 770 and h(Oradea) = 753 — the **wrong order**. Arad has three roads, so its probability is spread over Zerind and Timisoara too (“dilution”); Oradea has only two. A* then expands Oradea first (f = 71 + 753 = 824 vs 75 + 770 = 845) and finishes with **Zerind** → **Oradea** → **Sibiu** → **Fagaras** → **Bucharest**, cost **532** instead of 493. Letting the passer look at the receiver's threat (τ) is what turns this around.

Part 7 — The full table for goal = Bucharest and how to read it

city	true remaining cost d*	xGT	h (int)	h / d*	roads
Bucharest	0	1.0000	0	–	4
Urziceni	85	0.4088	89	1.05	3
Giurgiu	90	0.4066	90	1.00	1
Pitesti	101	0.3472	106	1.05	3
Hirsova	183	0.1447	193	1.05	2
Rimnicu Vilcea	198	0.1203	212	1.07	3
Fagaras	211	0.1117	219	1.04	2
Vaslui	227	0.0934	237	1.04	2
Craiova	239	0.0772	256	1.07	3
Eforie	269	0.0612	279	1.04	1
Sibiu	278	0.0489	302	1.09	4
Iasi	319	0.0352	335	1.05	2
Drobeta	359	0.0215	384	1.07	2
Neamt	406	0.0147	422	1.04	1
Arad	418	0.0108	453	1.08	3
Oradea	429	0.0097	464	1.08	2
Mehadia	434	0.0094	467	1.08	2
Zerind	493	0.0049	531	1.08	2
Lugoj	504	0.0044	542	1.08	2
Timisoara	536	0.0029	584	1.09	2

The true remaining cost d* is shown only to judge the heuristic; it is never used to compute it. Points to notice:

• The order is the same as the true order for every city (Spearman 1.000, zero inversions).

• h is 0–9 % above the true cost. Cities with one road (Giurgiu) or whose best road dominates completely pay nothing; cities with several plausible roads (Sibiu, Arad) pay the most. That is the “hesitation tax” of an imperfect passer.

• Cities in the same hop ring get different values: Fagaras (1 hop, 219) vs Giurgiu (1 hop, 90); Sibiu (2 hops, 302) vs Hirsova (2 hops, 193). The old ring method gave all cities in a ring the same value.

Why these parameter values

• **τ = 1.5.** With τ = 0 (the model as written in the football doc) the passer ignores where the ball is going; a three-road city like Arad spreads its probability over bad roads and is ranked behind two-road Oradea for goal Sibiu — a wrong answer. τ = 1 fixed the ranking; τ = 1.5 also removed the last non-optimal A* route (Zerind→Craiova) while h still differs from the road cost (Arad 151 vs true 140 for goal Sibiu). τ ≥ 2–3 makes the passer nearly perfect and h collapses onto the road cost — a renamed Dijkstra, which the guide rejects.

• **β = 0.02.** Slightly stronger preference for short roads than the doc's 0.01; it raised A* optimality without changing any ranking.

• **γ = 0.01.** Sets the unit: −ln(xGT)/γ is in road-cost units. Larger γ pulls h onto the road cost, smaller inflates it.

Result summary (all 380 start→goal pairs)

heuristic	guide pair checks	Spearman vs d*	A* optimal	nodes expanded
xGT (this)	8/8	0.999	380/380	4.94
UCS (h = 0)	0/8	–	380/380	11.00
straight-line distance	6/8	0.848	380/380	6.81
old ring / min-edge	4/8	0.975	380/380	5.60

Files: `final/method.py` (constants), `core/xgt.py` (Python formula), `tools/java/XgtHeuristic.java` (Java, identical output), `final/exports/heuristicTable.ts` / `HeuristicTable.java` (precomputed 20-goal integer tables for the app).